Unit II Assignment

1. **CSS Flexbox Navbar**

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .navbar {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 15px 30px;
      background-color: #333;
      color: white;
    }
    .logo {
      font-size: 24px;
      font-weight: bold;
    }
    .menu {
      display: flex;
      gap: 25px;
    }
    .menu a {
      color: white;
      text-decoration: none;
    }
  </style>
```

```
</head>
<body>
  <nav class="navbar">
    <div class="logo">MyLogo</div>
    <div class="menu">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Contact</a>
    </div>
  </nav>
</body>
</html>
```

2. Fix the Dark Mode Button Bug

Bug: addEventListner ki spelling galat hai. Correct method addEventListener() hai.



```
const button = document.querySelector('.toggle-btn');

button.addEventListener('click', () => {
    document.body.classList.toggle('dark-mode');
});
```

3. Signup Form Validation

```
const form = document.getElementById("signupForm");

form.addEventListener("submit", function(event) {
    event.preventDefault();

    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;

    if (!email.includes("@")) {
        alert("Please enter a valid email.");
        return;
    }

    if (password.length < 8) {
        alert("Password must be at least 8 characters.");
        return;
    }
```

4. Scroll Event Listener Memory Leak

Every time renderPage() runs, a new event listener is added. Old listeners remain attached, causing unnecessary memory usage and duplicate executions.

```
function handleScroll() {
    console.log(window.scrollY);
}

function renderPage() {
    window.removeEventListener('scroll', handleScroll);
    window.addEventListener('scroll', handleScroll);
}

renderPage();
renderPage();
```

5. **Debounced Search Input**

```
<input type="text" id="searchInput" placeholder="Search products">

<script>
    const searchInput = document.getElementById("searchInput");

    let timer;

    searchInput.addEventListener("input", function() {
        clearTimeout(timer);

        const query = searchInput.value;

        timer = setTimeout(() => {
            searchProducts(query);
        }, 400);
    });

    function searchProducts(query) {
```

6. Mobile Media Query

Original CSS breaks because the fixed width of 960px is larger than the available width on small screens.

```
.container {
    display: flex;
    width: 960px;
}

.card {
    width: 300px;
}

@media (max-width: 480px) {
    .container {
        width: 100%;
        display: flex;
        flex-direction: column;
    }

    .card {
        width: 100%;
        box-sizing: border-box;
    }
}
```

7. **Save and Restore Theme Using localStorage**

```
const savedTheme = localStorage.getItem("theme");

if (savedTheme) {
    document.body.classList.add(savedTheme);
}

function setTheme(theme) {
    document.body.classList.remove("light", "dark");

    document.body.classList.add(theme);

    localStorage.setItem("theme", theme);
}
```

8. **Prevent Form Refresh and Submit Using fetch()**

```
<form id="contactForm">
    <input type="email" name="email" placeholder="Enter Email">
    <button type="submit">Send</button>
</form>

<script>
    const form = document.getElementById("contactForm");

    form.addEventListener("submit", async function(event) {
        event.preventDefault();

        const formData = new FormData(form);

        const data = Object.fromEntries(formData.entries());

        try {
            const response = await fetch("/api/contact", {
                method: "POST",
```

9. Correct the Async Function

await is required before both fetch() and res.json().

```
async function getUser(id) {
    const res = await fetch(`/api/users/${id}`);
    const data = await res.json();

    return data;
}
```

10. Responsive CSS Grid Image Gallery

Question mein minmax() aur auto-fit use karna hai, media query nahi

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
            gap: 15px;
        }

        .gallery img {
            width: 100%;
            height: 200px;
            object-fit: cover;
            border-radius: 8px;
        }
    </style>
</head>
```